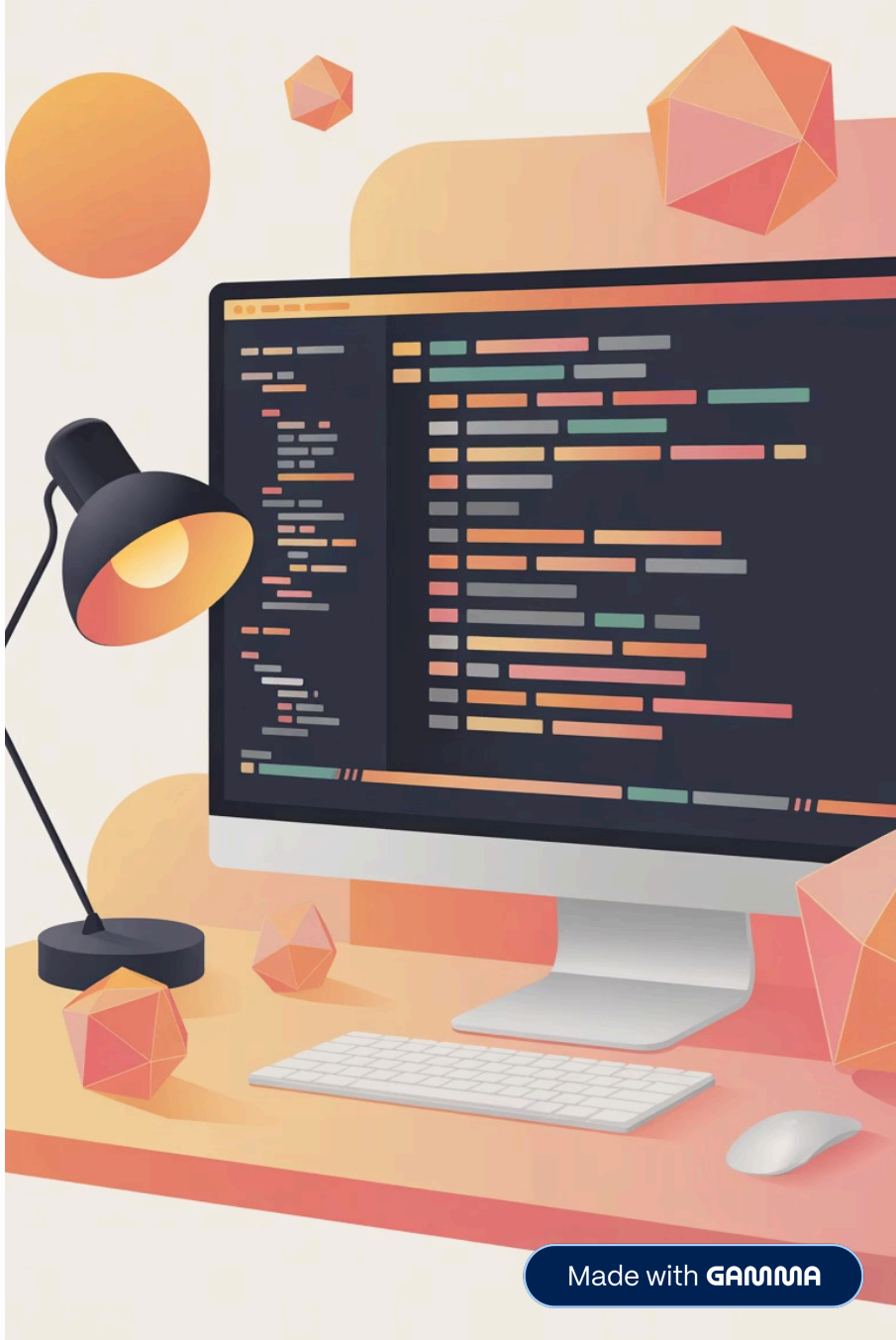
Lección 6: Funciones

En esta lección aprenderemos a crear bloques de código reutilizables que automatizan tareas y evitan la repetición innecesaria.

Seguiremos el principio fundamental **DRY** (**D**on't **R**epeat **Y**ourself), una filosofía esencial para cualquier analista de datos que busque escribir código limpio, mantenible y eficiente.

El objetivo es claro: escribir la lógica de limpieza o cálculo una sola vez y aplicarla a múltiples conjuntos de datos.



Made with GAMMA

Funciones: la estructura def

Una función es un bloque de código con un nombre asignado que solo se ejecuta cuando es **llamado explícitamente**. Este mecanismo nos permite encapsular lógica compleja y reutilizarla tantas veces como sea necesario, simplemente invocando su nombre.

La sintaxis utiliza la palabra clave **def**, seguida del nombre descriptivo de la función, paréntesis para los parámetros y dos puntos para iniciar el bloque de código indentado.

Definición

```
def calcular_iva(precio):
    # Cuerpo de la función
    return precio * 0.21
```

Declaramos la función con sus parámetros

Llamada

```
impuesto = calcular_iva(100)
```

Ejecutamos la función pasando argumentos necesarios

Resultado

La función procesa la entrada y devuelve un valor que podemos almacenar o utilizar en cálculos posteriores.

Made with GAMMA

Ámbito de Variables (Scope): Local vs. Global

Variable Local

Creada **dentro** de la función. Solo existe y es accesible dentro de ese bloque específico. Cuando la función termina su ejecución, esta variable desaparece de la memoria.

- Mayor control y aislamiento
- Previene conflictos de nombres
- Facilita el debugging

Variable Global

Creada **fuera** de cualquier función. Es accesible desde cualquier parte del código, incluido el interior de funciones (solo lectura por defecto).

- Disponible en todo el script
- Puede causar efectos secundarios
- Requiere keyword `global` para modificar

Regla del Analista

La función debe ser **autónoma y predecible**. Evita depender de o modificar variables globales dentro de la función (a menos que uses `global`), para asegurar que siempre dé el mismo resultado para la misma entrada. Esta práctica garantiza código más robusto y fácil de mantener.

Made with GAMMA

Devolviendo Resultados

La sentencia `return` es el mecanismo fundamental que permite a la función **enviar un valor de vuelta** a la línea de código que la llamó. Sin ella, la función puede ejecutar acciones, pero no proporciona resultados utilizables en cálculos posteriores.

Función sin return

Ejecuta el código (ejemplo: `print()`, guardar archivo), pero devuelve **None**, que no es útil para cálculos encadenados.

Función con return

Devuelve el resultado del procesamiento: números, listas, diccionarios, o *dataframes* completos.

Uso en Análisis

Imprescindible para el análisis de datos, ya que permite que los resultados se usen en el resto del *script* o se encadenen con otras funciones.

Made with GAMMA

Argumentos: Personalizando la Función

01

Argumentos Posicionales

Se asignan por el **orden** en que se definen en la función. Requieren precisión absoluta en la llamada, ya que el primer argumento va al primer parámetro, el segundo al segundo, y así sucesivamente.

02

Argumentos por Palabra Clave

Se asignan especificando explícitamente el nombre del parámetro (`nombre=valor`). **Ignoran el orden** de los parámetros, lo que mejora significativamente la **legibilidad** del código y reduce errores.

03

Valores por Defecto

Si el usuario no proporciona un valor para cierto parámetro, la función usa automáticamente el valor predefinido en la declaración.

```
def calculo(dato, factor=1.1): # 1.1 es el valor por defecto
 return dato * factor

# Llamadas válidas:
resultado1 = calculo(100) # Usa factor=1.1
resultado2 = calculo(100, 1.5) # Usa factor=1.5
resultado3 = calculo(dato=100, factor=1.2) # Argumentos por palabra clave
```

Made with GAMMA

BONUS: Argumentos Flexibles (*args y **kwargs)

Cuando el Número de Entradas es Variable

*args (Argumentos No Clave)

Permite pasar un número **variable** de argumentos **posicionales** a la función. Internamente, la función recibe estos valores como una **Tupla**, lo que facilita su iteración y procesamiento.

- Ideal para operaciones con cantidad desconocida de elementos
- Útil para sumar o promediar métricas variables
- Sintaxis: `def funcion(*args)`

**kwargs (Argumentos Clave)

Permite pasar un número **variable** de argumentos **por palabra clave** a la función. La función los recibe como un **Diccionario**, con nombres de parámetros como claves y sus valores correspondientes.

- Perfecto para configuraciones flexibles
- Útil en reportes con filtros opcionales
- Sintaxis: `def funcion(**kwargs)`

Ejemplo de *args

```
def sumar_metricas(*valores):
    return sum(valores)

total = sumar_metricas(10, 20, 30, 40)
# total = 100
```

Ejemplo de **kwargs

```
def filtrar_datos(**filtros):
 # filtros = {'pais': 'es',
 # 'ciudad': 'madrid',
 # 'anio': 2023}
 return filtros

filtrar_datos(pais="es", anio=2023)
```

Made with GAMMA

BONUS: Funciones Avanzadas: Lambda y Recursividad

1

Funciones Lambda

Pequeñas funciones **anónimas** definidas en una sola línea. No requieren la palabra clave `def` ni un nombre explícito.

2

Uso Práctico

Ideales para operaciones sencillas o como argumentos de funciones de orden superior como `map()`, `filter()` o `apply()`.

3

Recursividad

Una función que se **llama a sí misma** para resolver un problema dividiéndolo en subproblemas idénticos más pequeños.

Lambda: Sintaxis Compacta

```
calcular_doble = lambda x: x * 2

# Uso con map
valores = [1, 2, 3, 4, 5]
dobles = list(map(lambda x: x * 2, valores))
# dobles = [2, 4, 6, 8, 10]
```

Perfectas para transformaciones rápidas de datos sin necesidad de definir una función completa.

Recursividad: Factorial

```
def factorial(n):
    if n == 0 or n == 1:  # Condición de parada
        return 1
    return n * factorial(n - 1)

resultado = factorial(5)  # 120
```

¡Advertencia! Las funciones recursivas requieren una **condición de parada** clara para evitar bucles infinitos y el agotamiento de la pila de llamadas.

Made with GAMMA

Resumen

def

Codifica tu lógica **una vez** y reutilízala infinitamente. Elimina la duplicación de código y facilita el mantenimiento de tus scripts de análisis.

Scope

Mantén la lógica **local** para garantizar la predictibilidad. Las funciones autónomas son más fáciles de probar, depurar y reutilizar en diferentes contextos.

return

Devuelve los resultados del procesamiento para usarlos en análisis posteriores. Sin `return`, tus funciones no generan outputs aprovechables.

*args / **kwargs

Diseña funciones **generales** que acepten entradas flexibles. Aumenta la versatilidad de tu código y reduce la necesidad de múltiples versiones de la misma función.

Made with GAMMA